SHARING · DOMAIN 4 - DATA SHARING AND FEDERATION

Lakehouse Federation and its limits

Section 4. Query the data where it lives, and know what that costs.

SECTION 4 of 10 WEIGHT 5% TOPIC Federation

admin@sthithapragna.com

Federation puts an external database **inside your catalog**. Same three-level names, same grants, same lineage. The data never moves.

Two securables, and one more for catalog federation

OBJECT	WHAT IT HOLDS
Connection	The path and credentials for reaching an external system. A securable in its own right.
Foreign catalog	A mirror of an external database inside Unity Catalog, created from a connection.
External location	Only for catalog federation. Where the underlying files actually live.

ON THE EXAM

Creating a foreign catalog needs CREATE CATALOG on the metastore, plus either ownership of the connection or CREATE FOREIGN CATALOG on it.

Query federation against catalog federation

QUERY FEDERATION

Pushed to their database

- Translated to SQL and sent over JDBC
- Runs on Databricks and their compute
- For ad hoc reporting, BI, proof of concept
- Relational databases

CATALOG FEDERATION

Read from their storage

- Reads the files directly in object storage
- Runs on Databricks compute only
- More cost-effective and faster
- Catalog platforms: Hive, Glue, Snowflake

WORTH KNOWING

Catalog federation avoids the other system's compute entirely, which is why the docs call it more cost-effective and performance-optimized.

What each type connects to

TYPE	SOURCES
Query federation	MySQL, PostgreSQL, Oracle, Teradata, Redshift, Snowflake, SQL Server, Synapse, BigQuery, Salesforce Data 360, and Databricks.
Catalog federation	Legacy and external Hive metastores, AWS Glue, Snowflake, Salesforce Data 360, Palantir Foundry.

ON THE EXAM

Some sources appear in both lists. Snowflake and Salesforce can be reached either way, and the choice then follows the cost and performance argument.

The same shape, in four steps

- Create a **connection** holding the credentials and the address of the system.
- Create a **foreign catalog** from that connection. For catalog federation, also give it an external location.
- **Grant privileges** on the tables in the foreign catalog. This is where governance actually happens.
- Run queries. They are pushed down, or read from storage, depending on the type.

> ON THE EXAM
>
> Notice the third step. Registering the source is plumbing; granting access with Unity Catalog is the objective's 'proper governance'.

What Unity Catalog actually brings

- **Table-level access controls** on foreign tables, using ordinary grants.
- **Lineage**, so a dashboard's dependency on an external system is visible rather than folklore.
- **Search and discovery** alongside your own tables.
- Credentials live in the connection, so users query the source **without ever holding them**.

> ON THE EXAM
>
> That last point is the strongest argument. Federation replaces a connection string circulating in notebooks with a governed, revocable grant.

Federated queries are read-only

- You cannot write back to a federated source. Federation is a read path.
- The docs name one exception: catalog federation of a workspace's **legacy Hive metastore**, where foreign tables are writeable.
- If you need write access, the answer is the **Spark data source interface**, not federation.
- The documentation is not entirely consistent on the exception. Treat read-only as the rule.

> ON THE EXAM
>
> A requirement to update the external system rules federation out. That is usually the whole question.

Your query becomes their query

- Databricks translates your SQL into statements **pushed down** into the foreign source.
- Inspect the generated SQL from the **foreign data source scan node** in the Query Profile, or with EXPLAIN FORMATTED.
- Coverage varies by source. What cannot be pushed down is executed on Databricks after the data arrives.
- So a filter that does not push down means the whole table crosses the network first.

> ON THE EXAM
>
> When a federated query is slow, look at what actually got pushed down before blaming the source. The scan node tells you.

No caching, and their concurrency limits are yours

- **Databricks query caching is not supported** for federated queries. Neither result cache nor disk cache.
- So a repeated dashboard query hits the source every time, with no free second run.
- Throttling follows the Databricks SQL concurrent query limit, and you are competing with the source's own workload.
- A federated dashboard refreshed by fifty analysts is fifty queries against a production database.

> ON THE EXAM
>
> Federation makes an operational database look like a table. It does not make it behave like one under analytical load.

When Databricks says not to federate

- When a source supports **both federation and Lakeflow Connect**, Databricks recommends **Lakeflow Connect**.
- The stated condition: when performance at **higher data volumes** or **lower latency** is a priority.
- Federation is positioned for **ad hoc reporting, BI and proof of concept**, not for the production read path.
- That is a narrower recommendation than most material implies.

> ON THE EXAM
>
> Federation is a way to reach data, not a substitute for ingesting it. The docs are explicit about which is preferred at volume.

When to use the Spark data source interface

- When the source is **not supported** by Lakehouse Federation at all.
- When you need **write access** to the external system.
- When you need control over **execution and parallelisation**, such as partitioned reads.
- The runtime bundles connectors for PostgreSQL, SQL Server, MySQL, Snowflake and Redshift.

> ON THE EXAM
>
> Three reasons, each decisive on its own. Any of them makes federation the wrong tool rather than a slower one.

What to carry into the exam

- Connection plus foreign catalog. Governance is the **grants**, not the registration.
- **Query federation** pushes down over JDBC. **Catalog federation** reads storage on Databricks compute only.
- **Read-only**, **no caching**, and you inherit the source's concurrency limits.
- Databricks prefers **Lakeflow Connect** when volume or latency matter.
- Need to write, or an unsupported source? **Spark data source interface.**

> ON THE EXAM
>
> That completes Section 4, the smallest section at 5%. Section 5 is monitoring and alerting, at 10%.

> An analytics team federates a production PostgreSQL order system so analysts can query it without extracts. The proof of concept was fast. Six weeks after rollout, the database team reports sustained load and slow application response during business hours.
>
> A dashboard built on the foreign catalog is refreshed by around sixty analysts each morning.

What is the most likely explanation?

A Federated queries are not cached, so every refresh runs against the production database.

B The foreign catalog is holding a stale copy that needs refreshing more often.

C Query pushdown is disabled by default and must be enabled on the connection.

D The connection needs a higher concurrency limit configured in Unity Catalog.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Federated queries are not cached, so every refresh runs against the production database.

Neither result cache nor disk cache applies. Sixty refreshes are sixty queries on a production system.

B The foreign catalog is holding a stale copy that needs refreshing more often.

A foreign catalog holds no data. Nothing is copied.

C Query pushdown is disabled by default and must be enabled on the connection.

Pushdown is automatic. Coverage varies, but it is not a switch.

D The connection needs a higher concurrency limit configured in Unity Catalog.

Raising concurrency would increase load on the struggling database, not reduce it.

ON THE EXAM

A proof of concept that went well is the setup here. Federation scales differently from how it demonstrates.

> A team needs to read reference data from an on-premises system and also write status updates back to it after processing. The source is a supported Lakehouse Federation database.
>
> An architect proposes federating it and using MERGE to update the foreign tables.

What is wrong with the proposal?

A. Nothing. Foreign tables support MERGE like any Unity Catalog table.

B. Federated queries are read-only. Writing needs the Spark data source interface.

C. MERGE is supported but only through catalog federation, not query federation.

D. Foreign tables must be cloned into Unity Catalog before they can be written to.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A. Nothing. Foreign tables support MERGE like any Unity Catalog table.
Federation is a read path. Foreign tables are not writeable.

B. Federated queries are read-only. Writing needs the Spark data source interface.
Write access is one of the three documented reasons to choose the Spark data source interface instead.

C. MERGE is supported but only through catalog federation, not query federation.
Catalog federation is read-only too, apart from the legacy Hive metastore case.

D. Foreign tables must be cloned into Unity Catalog before they can be written to.
Cloning would write into Unity Catalog, not into the external system.

ON THE EXAM

A write requirement rules federation out on its own. It does not matter how well it would have handled the read.

> Data engineers currently paste a Snowflake connection string and service account password into each notebook that reads from it. A security review requires that no user holds those credentials, while analysts keep read access to two specific tables.
>
> The team is comfortable with the data staying in Snowflake.

What best meets the requirement?

A. Move the password into a Databricks secret scope and keep the current notebooks.

B. Create a connection and foreign catalog, then grant SELECT on the two tables.

C. Ingest both tables nightly with Lakeflow Connect and grant on the copies.

D. Use Delta Sharing to share the two Snowflake tables into Unity Catalog.

◆ PAUSE AND DECIDE

What best meets the requirement?

A. Move the password into a Databricks secret scope and keep the current notebooks.
Better than a literal, and anyone who can read the scope still holds the credential.

B. Create a connection and foreign catalog, then grant SELECT on the two tables.
Credentials live in the connection, so users query without ever holding them, and grants are table-level.

C. Ingest both tables nightly with Lakeflow Connect and grant on the copies.
Solves the credential problem by moving the data, which the team explicitly did not want.

D. Use Delta Sharing to share the two Snowflake tables into Unity Catalog.
Sharing works outward from a Databricks provider. It does not pull data in from Snowflake.

ON THE EXAM

This is the governance argument in its purest form: the credential stops being something a person holds and becomes something a grant confers.

> A federated query against a large Oracle table takes far longer than expected. The query filters on a date column and aggregates by region. The Oracle team reports seeing a full table scan.
>
> An engineer wants to know what Databricks actually sent.

How should they investigate?

A Enable query logging on the connection object in Unity Catalog.

B Open the Query Profile and read the foreign data source scan node, or run EXPLAIN FORMATTED.

C Check the Delta transaction log of the foreign table.

D Compare the initial and final plans to see what AQE changed.

◆ PAUSE AND DECIDE

How should they investigate?

A Enable query logging on the connection object in Unity Catalog.

Not a feature of connections.

B Open the Query Profile and read the foreign data source scan node, or run EXPLAIN FORMATTED.

Both show the statement actually pushed into the source, which tells you whether the filter went down.

C Check the Delta transaction log of the foreign table.

A foreign table is not a Delta table and has no transaction log.

D Compare the initial and final plans to see what AQE changed.

Right technique from module 13, wrong problem. AQE re-plans Databricks execution, not what is pushed down.

ON THE EXAM

A full table scan on their side alongside a filter on yours means the filter did not push down. The scan node is where you confirm it.

> A company is migrating from an external Hive metastore to Unity Catalog. The migration will take about a year, and during that time some tables must remain managed by the Hive metastore while new work happens in Unity Catalog.
>
> They want governed access to both from one place, at the lowest cost.

Which approach fits?

A Query federation against the Hive metastore, which pushes queries down over JDBC.

B Catalog federation, which reads the files directly in object storage using only Databricks compute.

C Migrate everything at once to avoid running two systems.

D Delta Sharing between the Hive metastore and Unity Catalog.

◆ PAUSE AND DECIDE

Which approach fits?

A Query federation against the Hive metastore, which pushes queries down over JDBC.

A Hive metastore holds metadata, not a query engine to push SQL into.

B Catalog federation, which reads the files directly in object storage using only Databricks compute.

The documented use case: incremental migration or a long-term hybrid, and cheaper because no external compute is involved.

C Migrate everything at once to avoid running two systems.

Ignores the stated constraint that some tables must remain for a year.

D Delta Sharing between the Hive metastore and Unity Catalog.

Sharing operates between Unity Catalog metastores, not from a Hive metastore.

ON THE EXAM

Catalog federation exists for exactly this shape of problem. Recognising incremental migration in a scenario is the cue.

> A team federates a Salesforce source for a dashboard. It works well for a monthly report over a small slice of data. The business now wants the same data feeding an hourly pipeline over several hundred million rows, with tight latency targets.
>
> The source is supported by both Lakehouse Federation and Lakeflow Connect.

What does Databricks recommend?

A Keep federation and increase the warehouse size to handle the volume.

B Use Lakeflow Connect, which Databricks recommends when volume or latency are priorities.

C Keep federation but enable result caching to reduce repeated load.

D Use catalog federation instead, since it is more performance-optimized.

◆ PAUSE AND DECIDE

What does Databricks recommend?

A Keep federation and increase the warehouse size to handle the volume.

The constraint is the source and the network, not Databricks compute.

B Use Lakeflow Connect, which Databricks recommends when volume or latency are priorities.

Stated directly: where a source supports both, prefer Lakeflow Connect when performance at volume or lower latency matters.

C Keep federation but enable result caching to reduce repeated load.

Caching is not supported for federated queries at all.

D Use catalog federation instead, since it is more performance-optimized.

True in general, and it is a different query path for catalog platforms rather than a faster federation setting.

ON THE EXAM

Federation is scoped to ad hoc reporting, BI and proof of concept. Growing past that scope is the signal to ingest instead.